Some general tips

It might be useful for the reader to have in mind the 8 bits of the register Rx_SC since it will be one of the registers analyzed afterwards. The following table sums up the relevant information:

POSITION (BIT)	NAME OF THE FIELD	MEANING OF THE FIELD	READ OR WRITE?
0 (LSB)	**Rx_Ready**	Rx Buffer ready to be read	R
1	**Rx_Drop**	Drop received Rx frame	W
2	**Rx_FrameError**	Error in received Rx frame	R
3	**Rx_AbortSignal**	Rx frame was aborted	R
4	**Rx_Overflow**	Rx buffer is full	R
5	**Rx_FCSen**	FCS error detection enables	W
6-7	**N/A**	Not applicable	R

The register in address 0x2 is Rx_SC (register described previously).

Bit:	7 - 6	5	4	3	2	1	0
Field:	N/A	Rx_FCSen	Rx_Overflow	Rx_AbortSignal	Rx_FrameError	Rx_Drop	Rx_Ready
Reset:	0x0	0	0	0	0	0	0
R/W:	RO	WO	RO	RO	RO	WO	RO

Table 2.5: RX status/control register (Rx_SC), address=*0x2*

The register in address 0x3 is Rx_Buff (register which behaves as a data buffer). This register has 8 bits (like the Rx_SC register).

Bit:	7 - 0
Field:	Rx data byte
Reset:	0x0
R/W:	RO

Table 2.6: RX data buffer (Rx_Buff), address=*0x3*

There is a task called: "ReadAddress" which receives two parameters: the first one is the address of the register that it is necessary to read (input parameter of the task) and the second one is the value contained in that address (output parameter of the task).

By using the task ReadAddress, it is possible to get the value of the register into an array of 8 bits called ReadData (remember that both Rx_SC and Rx_Buff have 8 bits).

For the following sections, you can also infer from the file "*Running compilation and simulation*". But in order to avoid misunderstandings, we have extracted these key information to make it easier for you to complete the part A of the term project.

Coding the VerifyAbortReceive task

The read flags in Rx_SC are checked according to the following values:

POSITION	**NAME OF THE FIELD**	MEANING OF THE FIELD	**Checked value**
0 (LSB)	**Rx_Ready**	Rx Buffer ready to be read	**0**
2	**Rx_FrameError**	Error in received Rx frame	**0**
3	**Rx_AbortSignal**	Rx frame was aborted	**1**
4	**Rx_Overflow**	Rx buffer is full	**0**

In addition, after an abort event occurs, the data buffer should be zero. This is also a specification for the VerifyAbortReceive task.

Coding the VerifyOverflowReceive task

The read flags in Rx_SC are checked according to the following values:

POSITION	**NAME OF THE FIELD**	MEANING OF THE FIELD	**Checked value**
0 (LSB)	**Rx_Ready**	Rx Buffer ready to be read	**1**
2	**Rx_FrameError**	Error in received Rx frame	**0**
3	**Rx_AbortSignal**	Rx frame was aborted	**0**
4	**Rx_Overflow**	Rx buffer is full	**1**

Coding the VerifyNormalReceive task

The read flags in Rx_SC are checked according to the following values:

POSITION	**NAME OF THE FIELD**	MEANING OF THE FIELD	**Checked value**
0 (LSB)	**Rx_Ready**	Rx Buffer ready to be read	**1**
2	**Rx_FrameError**	Error in received Rx frame	**0**
3	**Rx_AbortSignal**	Rx frame was aborted	**0**
4	**Rx_Overflow**	Rx buffer is full	**0**

In addition, the Rx_Buff should be read once more by using the ReadAddress task. A loop can be used, for each iteration, to check the value in ReadAddress corresponds to the byte stored in each row of the received matrix named **data** (note that each row in the mentioned structure stores one byte of data).

A pass will be inferred unless the read content in the buffer does not correspond to the iterated byte in the matrix. The number of iterations in the loop is controlled by using the input parameter “Size”.